

GRADUAÇÃO

 FIAP

MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 07

Protegendo o aplicativo

Telas protegidas e Firebase Auth



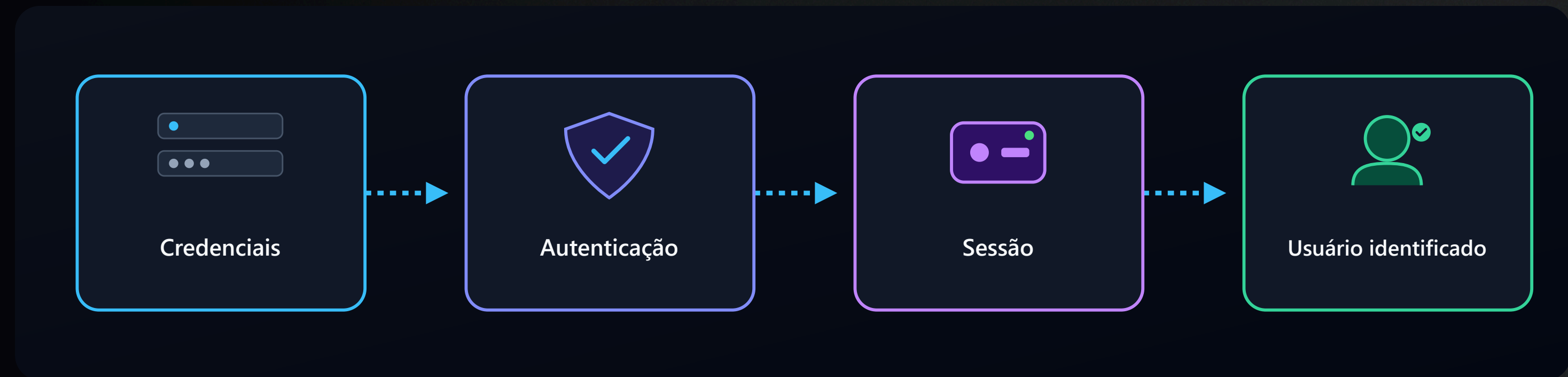
“Todo programa e todo usuário do sistema deve operar usando o menor conjunto de privilégios necessário para realizar a tarefa.”

— Jerome H. Saltzer e Michael D. Schroeder



O APLICATIVO JÁ SABE AUTENTICAR

Já temos um fluxo capaz de:



AGORA ESSA IDENTIDADE PRECISA INFLUENCIAR O RESTANTE DO APLICATIVO.

NEM TODA TELA É PÚBLICA

UM APLICATIVO PODE MISTURAR ÁREAS ABERTAS E ÁREAS PESSOAIS.

ACESSO LIVRE

Conteúdo público

Qualquer pessoa acessa

PROTEGIDO

Conteúdo privado

Somente usuário autenticado

A IDENTIDADE CONTROLA O ACESSO

O ESTADO DA AUTENTICAÇÃO PASSA A DECIDIR QUAIS PARTES DO APLICATIVO FICAM DISPONÍVEIS.



PACEUP

Nesta aula vamos construir o PaceUp, um aplicativo para descobrir corridas de rua, consultar detalhes das provas e acessar uma área pessoal com as inscrições do usuário.

QUALQUER PESSOA PODE EXPLORAR OS EVENTOS.

Eventos



Detalhes

A ÁREA PESSOAL EXIGE UMA IDENTIDADE.

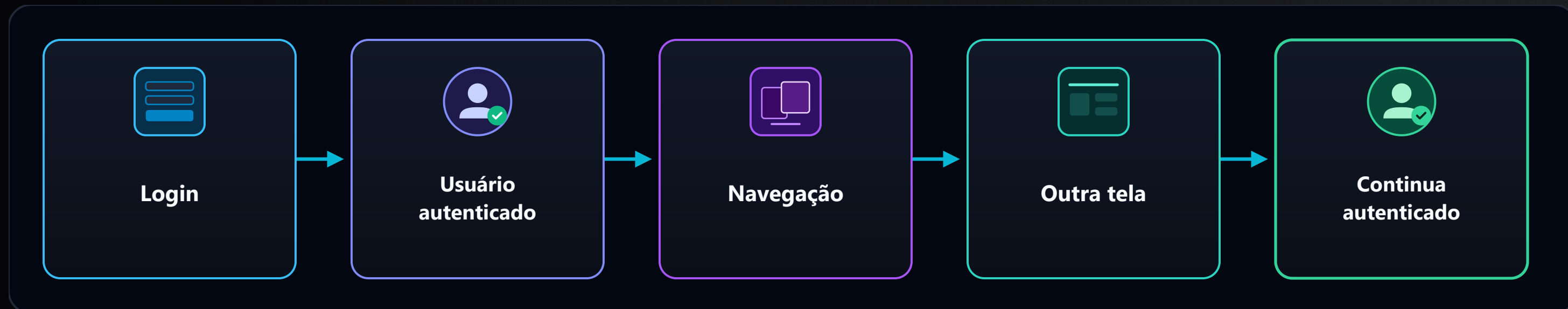
Minhas inscrições



usuário autenticado

A AUTENTICAÇÃO PRECISA PERMANECER DISPONÍVEL

Depois do login, o aplicativo precisa continuar sabendo quem é o usuário.



ESSE ESTADO PRECISA SER MANTIDO E OBSERVADO.

PODEMOS USAR UM SERVIÇO DE IDENTIDADE

Autenticação envolve mais do que validar email e senha.

Um sistema de identidade precisa lidar com:

- **Usuários e credenciais**
- **Sessões e persistência**
- **Recuperação de acesso**
- **Verificação de identidade**

Parte dessa infraestrutura pode ser delegada a um serviço especializado.

FIREBASE AUTHENTICATION

Firebase Authentication é um serviço para gerenciar identidade e sessão de usuários.

Nesta aula, ele será responsável por:

- criar usuários;
- autenticar credenciais;
- manter a sessão;
- informar mudanças no estado de autenticação.

CONTA E PROJETO FIREBASE

O Firebase Console é acessado com uma conta Google.

Dentro dele criamos um projeto.

```
conta Google
  ↓
Firebase Console
  ↓
projeto Firebase
```

O projeto agrupa os serviços usados pelo aplicativo.

REGISTRO DO APLICATIVO

Dentro do projeto Firebase, registramos o aplicativo.

```
projeto Firebase
  ↓
registrar app Web
  ↓
firebaseConfig
```

○ `firebaseConfig` identifica qual projeto será usado pelo código.

AUTHENTICATION HABILITADO

Criar o projeto não ativa login automaticamente.

Firebase Console



Authentication



Sign-in method



Email/Password

Nesta aula usaremos somente email e senha.

INSTALAÇÃO

```
npx expo install firebase
```

Usaremos Firebase JavaScript SDK com Expo.

CONFIGURAÇÃO

```
// lib/firebase.ts

import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: "...",
  authDomain: "...",
  projectId: "...",
  appId: "...",
};

const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
```

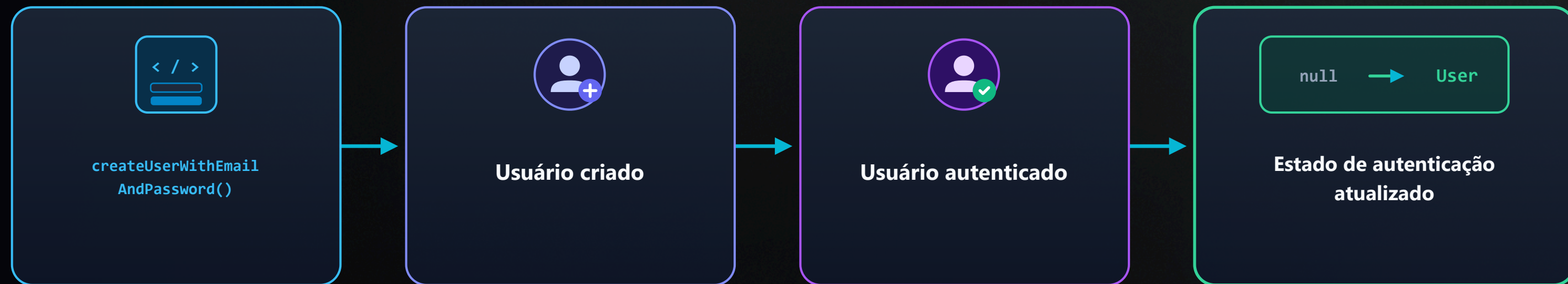
CRIAR CONTA

```
import { createUserWithEmailAndPassword } from "firebase/auth";
import { auth } from "@lib/firebase";

await createUserWithEmailAndPassword(auth, email, password);
```

Criar conta também autentica o usuário.

ESTADO APÓS O CADASTRO



ENTRAR

```
import { signInWithEmailAndPassword } from "firebase/auth";
import { auth } from "@lib/firebase";

await signInWithEmailAndPassword(auth, email, password);
```

ENTRAR USA UMA CONTA JÁ EXISTENTE.

OBSERVAR AUTENTICAÇÃO

```
import { onAuthStateChanged } from "firebase/auth";

onAuthStateChanged(auth, (user) => {
  setUser(user);
  setIsLoading(false);
});
```

O aplicativo reage ao estado do Firebase.

ESTADO INICIAL

AO ABRIR O APP, A SESSÃO AINDA ESTÁ SENDO VERIFICADA.



Durante esse intervalo, usamos `isLoading`.

AUTHPROVIDER

```
AuthProvider
├── user
├── isLoading
├── signUp()
├── signIn()
└── signOut()
```

O provider concentra o estado e as operações de autenticação.

RESPONSABILIDADES

`lib/firebase.ts`

→ configura o Firebase

`providers/AuthProvider.tsx`

→ mantém o estado da autenticação

`app/_layout.tsx`

→ protege as rotas

`sign-up / sign-in`

→ executam autenticação

ROTAS DO PACEUP

```
app/  
├── index.tsx  
├── event/  
│   └── [id].tsx  
├── sign-up.tsx  
├── sign-in.tsx  
└── registrations.tsx
```

PÚBLICO E PRIVADO

```
públicas
├── index
├── event/[id]
├── sign-up
└── sign-in

+
|
privada
└── registrations
```

STACK.PROTECTED

```
<Stack>
  <Stack.Screen name="index" />
  <Stack.Screen name="event/[id]" />

  <Stack.Protected guard={!user}>
    <Stack.Screen name="sign-up" />
    <Stack.Screen name="sign-in" />
  </Stack.Protected>

  <Stack.Protected guard{!!user}>
    <Stack.Screen name="registrations" />
  </Stack.Protected>
</Stack>
```

REGRA DE ACESSO

sem usuário



eventos

detalhes

cadastro

login

com usuário



eventos

detalhes

minhas inscrições

FLUXO COMPLETO

```
criar conta
  ↓
  Firebase Auth
    ↓
  onAuthStateChanged recebe User
    ↓
  AuthProvider atualiza user
    ↓
  Stack.Protected libera registrations
```

LOGOUT

```
import { signOut } from "firebase/auth";  
import { auth } from "@lib/firebase";  
  
await signOut(auth);
```

```
signOut()  
  ↓  
onAuthStateChanged recebe null  
  ↓  
registrations volta a ficar protegida
```



APLICATIVO “PACEUP”

Identidade e rotas protegidas com Firebase Auth

[Acesse o Codelab](#)

O QUE VOCÊ VAI APRENDER?

- configurar um projeto no Firebase;
- criar contas com email e senha;
- observar o estado de autenticação;
- manter a sessão no aplicativo;
- proteger telas com `Stack.Protected`.

REFERÊNCIAS

- Expo Router — Protected Routes
<https://docs.expo.dev/router/advanced/protected/>
- Expo — Using Firebase
<https://docs.expo.dev/guides/using-firebase/>
- Firebase Auth — Web Start
<https://firebase.google.com/docs/auth/web/start>
- Firebase Auth — Manage Users
<https://firebase.google.com/docs/auth/web/manage-users>

RECOMENDAÇÃO DE LEITURA

Muitos sistemas modernos evitam implementar toda a lógica de identidade e autorização por conta própria, adotando protocolos e serviços especializados para controlar acesso.

OAuth 2.0: Proteja suas aplicações com o Spring Security OAuth2 – Adolfo Eloy

✨ Por que ler?

-  Apresenta padrões usados para delegar autorização entre aplicações
-  Ajuda a entender o papel de tokens no controle de acesso
-  Introduz conceitos relacionados a OAuth 2.0, JWT e OpenID Connect



LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026